

FUNCTIONS AND RECURSION

Function (Subroutine or subprogram)

- ✓ “Function is a small program or program segment that carryout some specific well-defined tasks”.

Advantages of Functions

- Reduces the Complexity.
 - ✓ When a program contains more number of instructions (complex program), then such large program can be divided into number of sub programs called as functions.
- Improves the Readability.
- Easy to debug the errors.
- Reusability: The set of instructions specifying the task performed by the function is written just once but it can be used any number of times.
- Easy to do the modifications.

Types of Functions

i. Library Functions/Pre-Defined/ Built-in Functions

- ✓ C Library of C Compiler has a collection of various functions which perform some standard and pre-defined tasks.
- ✓ These functions written by designers of C Compilers are called as Library functions/Pre-Defined/Built-in functions.

Ex: sqrt(n)- computes square root of n.

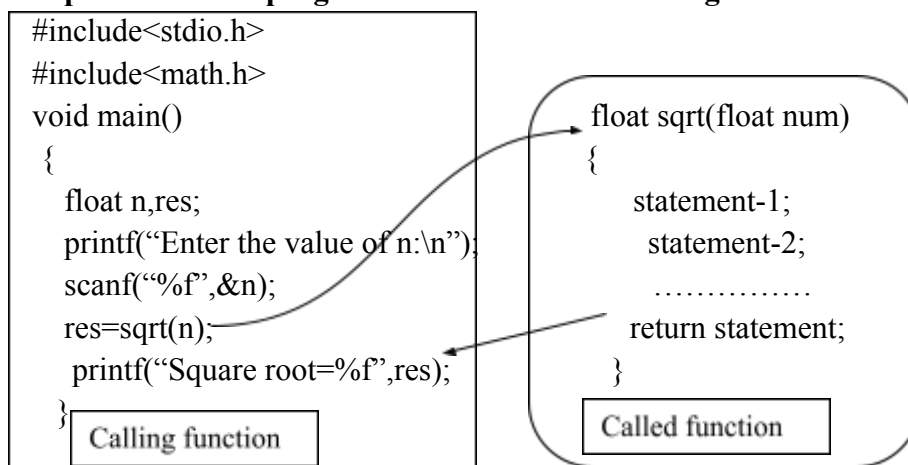
pow(x,y)- computes x^y .

printf()- used to print the data on the screen.

scanf()- used to read the data from the keyboard.

abs(x)- computes absolute value of x.

Example: Write a C program to demonstrate the usage of Built-in functions.



Called Function and Calling Function

- ✓ A function that is called (invoked) by writing the name of the function is called “Called Function”.
- ✓ The invoking function is called “Calling Function”.

Example: in the above program main() function invokes sqrt() function, the function main() is calling function and function sqrt() is called function.

ii. User-Defined/ Programmer Defined Functions

- ✓ The functions written by the programmer/user to do the specific tasks is called User-Defined/ Programmer Defined Functions.
- ✓ main() is the user defined function.

Elements of User-Defined Functions

- ✓ The three elements of user-defined functions are shown below:
 - i. Function Prototype/Declaration
 - ii. Function Definition
 - iii. Function Call

Function Prototype/Declaration

- ✓ As we normally declare the variables before they are used, the functions also should be declared before they are used.
- ✓ The process of declaring the functions before they are used (or called) in the program is called Function Prototype.
- ✓ The function prototype is also called as Function declaration.
- ✓ It is same as the Function Header but terminated with semicolon.
- ✓ It does not contain the body of the function.

Syntax

```
return_type function_name(parameter list);
```

Where,

return_type: This is the data type of the value that the function is expected to return (int, float, double, char).

function_name: It is the name of the function. It can be any valid Identifier.

parameter list: The parameters are the list of variables enclosed within parenthesis. Variables are separated by comma.

Ex: `int add(int a,int b);`

- ✓ The function prototype contains the following information in a single line terminated by semicolon:
 - The type of the value returned by the function.
 - The name of the function.
 - The number of parameters passed to that function.
 - The type of each parameter.

Function Definition

- ✓ The program module that is written to achieve a specific task is called **function definition**.
- ✓ Each function definition consists of two parts:

Function Header

Function Body

Syntax:

```
return_type function_name (parameter list)
{
    declaration part;
    executable part;
    return statement;
}
```

return_type:

- ✓ This is the data type of the value that the function is expected to return (int, float, double, char).
- ✓ If the function is not returning any value, then we need to specify the return type as 'void'.
- ✓ The default return value of any function is integer.

function_name:

- ✓ It is the name of the function. It can be any valid Identifier.

Parameters:

- ✓ The parameters are the list of variables enclosed within parenthesis. Variables are separated by comma.

Ex: int add (int a, int b);

return type Function name Data type of parameters Parameter name

return type Function name

Function Body

- ✓ **Declaration part:** All the variables used in the function body should be declared in this part.
- ✓ **Executable par:** This part contains the statements or instructions that perform the specified activity.
- ✓ **return statement:** It is a keyword used to return the control to the calling function (main) with/without a value.

Syntax:

1.

```
return;
```

Returns control without sending any value to main program

2.

```
return expression;
```

Returns control by sending value to main program

Ex: int add(int a,int b)

```
{
    int sum;      //variable declaration
    sum=a+b;     //executable statement
    return sum;  //return statement
}
```

Function Call

- ✓ Once the function is defined, it has to be called so as to achieve the task. This method of calling a function to achieve a specified task is called **Function Call**.
- ✓ The function can be called by writing the name of the function and passing the appropriate number of arguments.
- ✓ The number of arguments in the function call and number of parameters in the function definition must match.
- ✓ Also the order of arguments in the function call and parameters in the function definition must match.

Example: add(m,n);

Example Program: Write a C program to perform the addition of two numbers using function

```
#include<stdio.h>
int add(int a,int b);      /*Function Prototype*/
int add(int a,int b)
{
    int sum;
    sum=a+b;
    return sum;
}
void main()
{
    int result;
    result=add(10,20);
    printf("sum=%d\n",result);
    getch();
}
```

1. The function add() is called with two arguments 10 and 20.
2. The control is transferred to the function add() and the values 10 and 20 are received using variables a and b.
3. The result is computed by adding 10 and 20.
4. The result is returned to the main() and will be copied into the variable result.
5. The result is displayed on the screen.

- ✓ The 'main()' is always the "Calling Function".
- ✓ The 'add()' function is called "Called Function".

Function Parameters

- ✓ “The list of variables defined in the function header within the parenthesis are called **Function parameters**”.
- ✓ There are 2 types of parameters in ‘C’ functions.
 - i. Actual parameters
 - ii. Formal parameters

i. Actual or Real Parameters

- ✓ **The variables that are used when a function is invoked are called actual parameters.**
- ✓ Actual parameters are used in the Calling function when a function is invoked.
- ✓ Actual parameters send values or addresses to the formal parameters. Formal parameters receive them and use the same values.
- ✓ Actual parameters can be constants, variables or expressions.

Ex: res = add (m, n);

ii. Formal or Dummy Parameters

- ✓ **The variables defined in the function header or function definition are called formal parameters.**
- ✓ All the variables should be separately declared and each declaration must be separated by commas.
- ✓ The formal parameters receive values from the actual parameters.
- ✓ If the formal parameters receive the address from the actual parameters, then they should be declared as pointers.
- ✓ The formal parameters should be only variables. Expressions and constants are not allowed.

Ex: int add(int a,int b);

Example Program: C Program to define actual and formal parameters.

```
#include<stdio.h>
```

```
int add(int a,int b)                // Formal Parameters a, b
{
    int sum;
    sum=a+b;
    return sum;
}
void main()
{
    int m,n,res;
    printf("Enter the values for m,n\n");
    scanf("%d%d",&m,&n);
    res=add(m,n);                   // Actual parameters m,n
    printf("Sum=%d\n",res);
}
```

Location of Functions

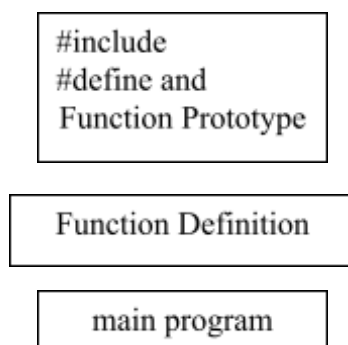
- ✓ The placement of the function definition in relation to the main program is very important.
- ✓ There are a number of ways to arrange the main program and a function.

i. Functions immediately after #includes and #defines

- ✓ In this method, we can place the entire the function definition consisting of the function header and function body in the beginning of the file immediately after #includes and #defines if any.
- ✓ The definition of the function appears before the main programs call to the function. This is called Top-down approach.
- ✓ In this situation, the function prototype is optional.

Syntax

file method1.c



- ✓ File method1.c contains a function followed by a main program.

Example Program

```
#include<stdio.h>
#include<conio.h>
int add(int a,int b);

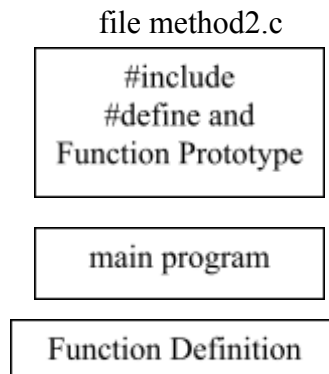
int add(int a,int b)
{
    int sum;
    sum=a+b;
    return sum;
}

void main()
{
    int m,n,res;
    printf("Enter the values for m,n:\n");
    scanf("%d%d",&m,&n);
    res=add(m,n);
    printf("add(%d,%d)=%d\n",m,n,res);
    getch();
}
```

ii. Functions after main

- ✓ In this method, immediately after #includes and #defines, we write all function prototypes.
- ✓ Next, we write the main function that is followed by the function definitions.

- ✓ Even though the function header comes after the main program, the function prototype appears before the call to the function and it is called Bottom-up approach.
- ✓ Function prototype is compulsory in this method.

Syntax:

- ✓ File method2.c contains a main program followed by a function.

Example Program

```
#include<stdio.h>
int add(int a,int b);
void main()
{
    int m,n,res;
    printf("Enter the values for m,n:\n");
    scanf("%d%d",&m,&n);
    res=add(m,n);
    printf("add(%d,%d)=%d\n",m,n,res);
}
```

```
int add(int a,int b)
{
    int sum;
    sum=a+b;
    return sum;
}
```

iii. Functions in Separate files (Separate Compilation)

- ✓ In this method, immediately after #includes and #defines we write all function prototypes.
- ✓ Next, we write the main function.
- ✓ All user function definitions will be in separate files.
- ✓ It puts the function in one file and the main program in another and compiles them separately.

This is the way library functions are used.

Syntax:

file method3.c

```
#include
#define and
Function Prototype
```

main program

file method4.c

Function Definition

separate file

- ✓ Here the file method3.c contains a main program and also a separate file.
- ✓ The file method4.c is a separate file containing a function and it performs some specific tasks and returns the value or result to the main program in a file 'method3.c'. i.e., a separate file.

Example Program

```
#include<stdio.h>
#include<conio.h>
int add(int a,int b);

void main()
{
    int m,n,res;
    printf("Enter the values for m,n:\n");
    scanf("%d%d",&m,&n);
    res=add(m,n);
    printf("add(%d,%d)=%d\n",m,n,res);
}
```

main.c

```
int add(int a,int b)
{
    int sum;
    sum=a+b;
    return sum;
}
```

add.c

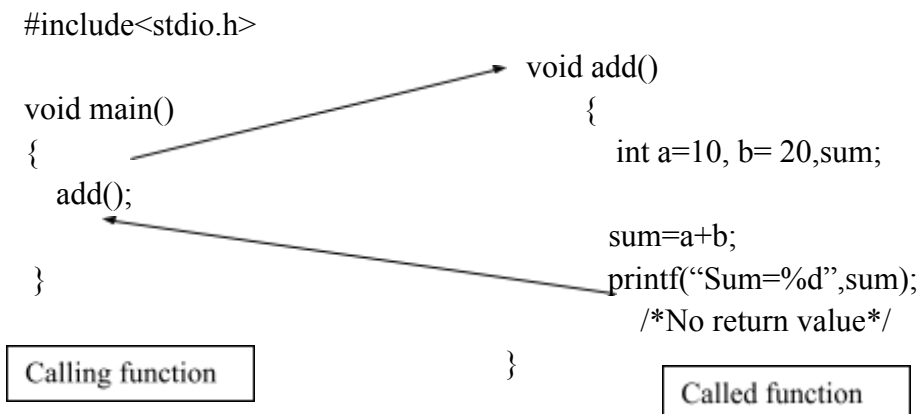
Categories of Functions

- ✓ Based on the parameters and return value, the functions have been classified into four categories.
1. **void and parameter less functions (Functions with no parameters and no return values)**
 2. **non void and parameter less functions(Functions with no parameters and return values)**
 3. **void with parameters functions (Functions with parameters and no return values)**
 4. **non void with parameters functions (Functions with parameters and return values)**

1. void and parameter less functions (Functions with no parameters and no return values)

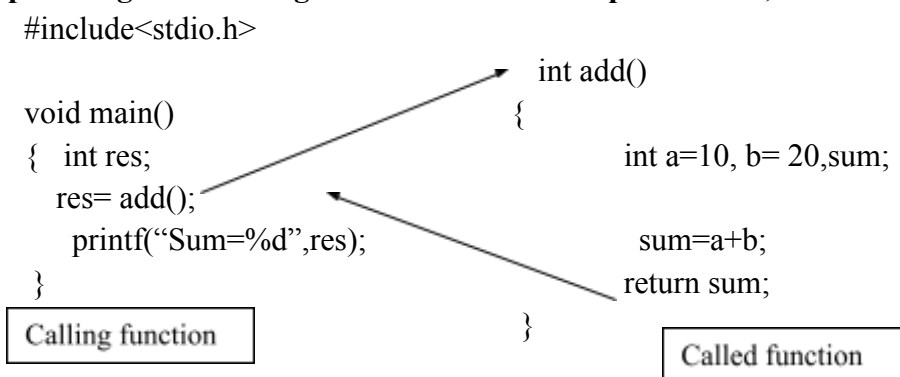
- ✓ In this category there is no data transfer between the calling function and the called function.
- ✓ So calling function cannot send values and hence called function cannot receive the data and it will not send any parameter back to function.

Example: Program showing function call with no parameters and returns no value.

**2. non void and parameter less functions (Functions with no parameters and return values)**

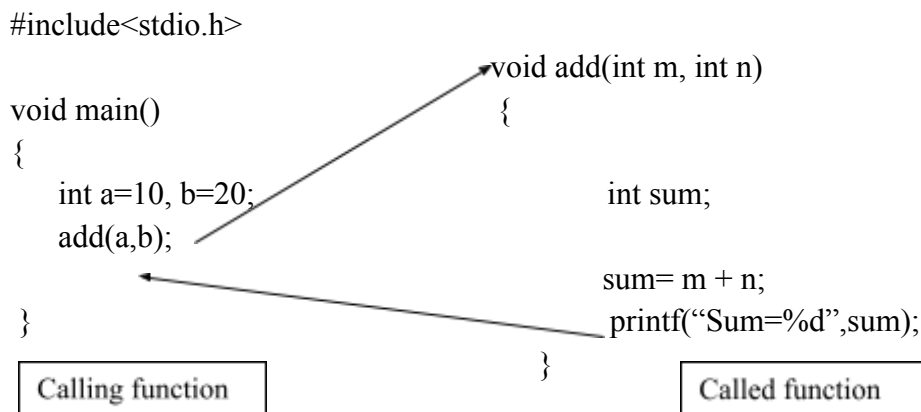
- ✓ In this category there is no data transfer from the calling function and the called function.
- ✓ But, there is data transfer from the called function to the calling function.
- ✓ When the function returns a value, the calling function receives one value from the called function.
- ✓ We can write a function that has no parameters but returns an answer to the main program as shown in the syntax.

Example: Program showing function call without parameters, with return value.

**3. void with parameters functions (Functions with parameters and no return values)**

- ✓ In this category, there is a data transfer from the calling function to the called function using parameters.
- ✓ But, there is no data transfer from the called function to the calling function

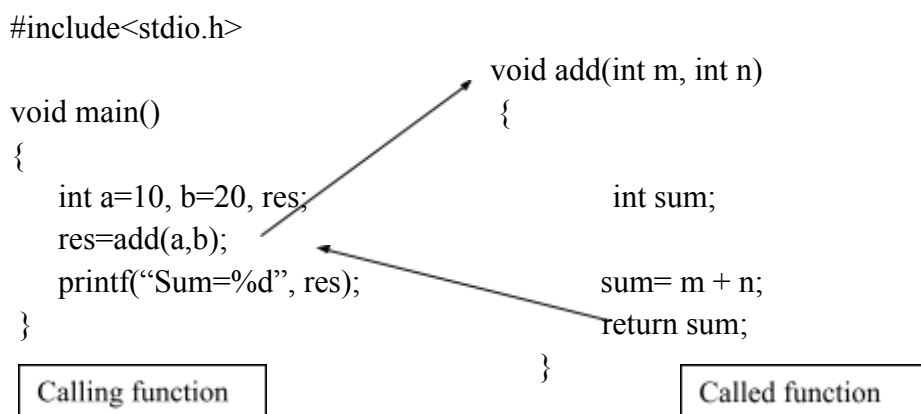
Example: Program showing a function call with parameters and no return value.



4. non void with parameters functions (Functions with parameters and return values)

- ✓ In this category, there is data transfer between the calling function and the called function.
- ✓ When parameters are passed, the called function can receive values from the calling function.
- ✓ When the function returns a value, the calling function can receive a value from the called function.

Example: Program showing a function call with parameters and with returns value.



Argument Passing or Parameter Passing Methods

- ✓ There are mainly two parameter passing mechanisms in 'C' functions. They are:

i. Call by Value or Pass by Value

ii. Call by Address or Pass by Address

i. Call by Value or Pass by Value

- ✓ Calling (Invoking) a function by passing values or variables to the function is called as call by value.
- ✓ The values of actual parameters are **copied** into formal parameters.
- ✓ Formal parameters contain only the copy of actual parameters. So, even if the values of the formal parameters changes in the called function, the values of the actual parameters are **not changed**.

Example Program: Write a C program to add two numbers using call by value.

```
#include<stdio.h>
```

```
void add (int a, int b)           // Formal parameters  a = 10, b =20
```

```
{
```

```
    int sum;
```

```
    sum = a + b;
```

```
    return sum;
```

```
}
```

```
void main()
```

```
{
```

```
    int m=10,n=20, res;
```

```
    res = add(m,n);           // Actual parameter m=10, n =20
```

```
    printf("result = %d \n", res);
```

```
}
```

Output:

Sum = 30

ii. Call by Address or Pass by Address

- ✓ **Call by address is done indirectly by passing the address of the variables in the function call and changing the value of the variable through its address.**
- ✓ In the called function, the formal parameters should be declared as pointers with the same type as the actual parameters.
- ✓ The addresses of actual parameters are copied into formal parameters. Using these addresses the values of the actual parameters can be changed.
- ✓ **In call by reference if any change in formal parameters imply then there is a change in actual parameters.**

Example Program: Write a C program to add two numbers using call by reference.

```
#include<stdio.h>
```

```
void add (int *a, int *b)       // Formal parameters  *a = 10, *b =20
```

```
{
```

```
    int sum;
```

```
    sum = *a + *b;
```

```
    return sum;
```

```
}
```

```
void main()
```

```
{
```

```
    int m=10,n=20, res;
```

```
    res = add(&m, &n);       // Actual parameter m=10, n =20
```

```
    printf("result = %d \n", res);
```

```
}
```

Output:

sum=30

Example Program: Write a C program to swap two numbers using call by reference.

```
#include<stdio.h>
void swap(int *a, int *b)
{
    int temp;
    temp=*a;
    *a=*b;
    *b=temp;
}
void main()
{
    int m=10,n=20;
    swap(&m,&n);
    printf("m=%d\n n=%d\n",m,n);
}
```

Output:

m = 20
n = 10

Using Arrays with Functions

The arrays can be passed to functions using two methods:

- i. Passing individual elements of the array
- ii. Passing the whole array

1. Passing individual elements of the array

✓ All array elements can be passed as individual elements to a function like any other variable.

Example program: Write a C program to print square of given array elements.

```
#include<stdio.h>
void print_square(int x)
{
    printf("%d", x*x);
}
void main()
{
    int n,a[10],i;
    printf("Enter the number of elements:");
    scanf("%d",&n);
    printf("\nEnter the array elements:\n");
    for(i=0;i<n;i++)
    {
        scanf("%d",&a[i]);
    }
    printf("Squares of array elements are:\n");
    for(i=0;i<n;i++)
    {
        print_square(a[i]); //Passing each individual element of the array to the function
    }
}
```

2. Passing the whole array

- ✓ Suppose, we want to pass whole array to a function. In such situation, we must follow the two rules:
- 1. The function must be called by passing only the name of the array.
- 2. In the function definition, the parameter must be declared as an array of the same type as that of actual parameter. There is no need to specify the size of the array.

Example Program: Write a C program to read the array elements using functions.

```
#include<stdio.h>

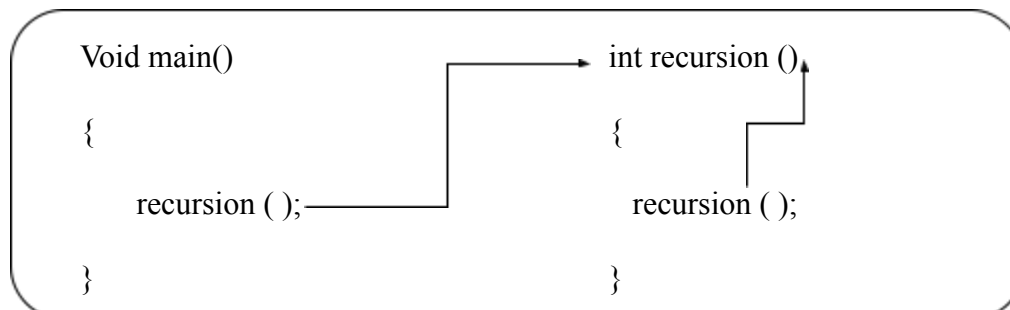
void read_array(int a[ ], int n)    // no need to specify the size of the array
{
    int i;
    for(i=0;i<n;i++)
    {
        scanf("%d",&a[i]);
    }
}

void main()
{
    int n,b[10],i;
    printf("Enter the number of elements:");
    scanf("%d",&n);
    printf("\nEnter the array elements:\n");
    read_array (b, n); // function must be called by passing only the name of the array
    printf("The array elements are:\n");
    for(i=0; i<n; i++)
    {
        printf("%d", b[i]);
    }
}
```

Recursion

- ✓ “The process in which a function calls itself again and again is called as Recursion”.
- ✓ A function which calls itself again and again is called as Recursive function.
- ✓ While using recursion, user need to be careful to define exit condition from function; otherwise it will go in infinite loop.
- ✓ Recursive functions are very useful to solve many mathematical problems like to calculate factorial of a number, generating Fibonacci series, etc.

Syntax:



Example: Program to calculate factorial of a given number.

```

#include<stdio.h>
int factorial(int n)
{
    if(n==1)
        return 1;
    else
        return (n*fact(n-1));
}
void main()
{
    int n,fact;
    printf("Enter a number=");
    scanf("%d",&n);
    fact=factorial(n);
    printf("\nFactorial of given number=%d",fact);
}

```

Output:

```

Enter a number=5
Factorial of given number =120

```

Example: Program to calculate nCr value using recursion function.

```

#include<stdio.h>
int factorial(int n)
{
    if(n==1)
        return 1;
    else
        return (n*fact(n-1));
}
void main()
{
    int n,fact;
    printf("Enter a value for n and r");
    scanf("%d",&n);

```

Output:

```

Enter a value for n and r
6
2
nCr Value =15

```

```
fact=factorial(n) / (factorial(n-r) * factorial(n));  
printf("\n nCr value =%d",fact);  
}
```

Example: C Program to find the Fibonacci series using recursive function.

```
#include<stdio.h>  
int fibonacci (int n)  
{  
    if( n == 0)  
        return 0;  
    else if (n == 1)  
        return 1;  
    else  
        return ( fibonacci (n-1) * fibonacci (n-2));  
}  
  
void main()  
{  
    int n, i, result;  
    printf("Enter a value for n ");  
    scanf("%d", &n);  
    printf(" The Fibonacci series is: \n");  
    for ( i =0; i < n ; i ++)  
    {  
        res = fibonacci (i);  
        printf("%d\n", res);  
    }  
}
```

Output:

Enter a value for n

5

The Fibonacci series is:

0

1

1

2

3

ASSIGNMENT QUESTIONS

1. Define function? Explain the declaration of function with example?
2. Explain the different Parameter used in function with example?
3. Explain the categories of functions with examples.
4. Explain the two parameter passing techniques used in functions.
5. Explain how to pass an array to function with example.
6. Define recursion? Explain the recursion with example.
7. Write a program to calculate nCr value using recursion.
8. Write a program to print the Fibonacci series using recursion.
9. Write a program to convert binary to decimal using recursion.
10. Write a C program to find factorial a given number using recursion.
11. Write a C program to calculate gcd and lcm of 2 integers using functions.